МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №1

Выполнил:
Студентка группы ИУ5-35Б
Шагина Д. А.
Преподаватель:
Гапанюк Ю. Е.

Нардид А. Н.

Москва 2026

Задание

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия: 1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко многим. Пример классов данных для предметной области Сотрудник-Отдел: 1. Класс «Сотрудник», содержащий поля:

- ID записи о сотруднике;
- Фамилия сотрудника;
- Зарплата (количественный признак);
- ID записи об отделе. (для реализации связи один-ко-многим) 2.

Класс «Отдел», содержащий поля:

- ID записи об отделе;
- Наименование отдела. 3. (Для реализации связи многие-ко многим) Класс «Сотрудники отдела», содержащий поля:
- ID записи о сотруднике;
- ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом.

Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Листинг кода

Main.py

```
class Schoolchild:
    def __init__(self, child_id, last_name, gpa, class_id):
        self.id = child_id
        self.last_name = last_name
        self.gpa = gpa
        self.class_id = class_id


class SchoolClass:
    def __init__(self, class_id, name):
        self.id = class_id
        self.name = name


class SchoolchildClass:
    def __init__(self, child_id, class_id):
        self.child_id = child_id
        self.class_id = class_id


classes = [
    SchoolClass(1, "Айти-класс"),
    SchoolClass(2, "Био-класс"),
    SchoolClass(3, "Академический"),
    SchoolClass(4, "Гуманитарный")
]

schoolchildren = [
    Schoolchild(1, "Иванов", 4.8, 1),
    Schoolchild(2, "Петров", 3.9, 1),
    Schoolchild(3, "Сидоров", 4.2, 2),
    Schoolchild(4, "Кузнецова", 4.5, 3),
    Schoolchild(5, "Смирнов", 3.5, 4),
    Schoolchild(6, "Попов", 4.7, 3)
]

schoolchildren_classes = [
    SchoolchildClass(1, 1),
    SchoolchildClass(1, 3),
```

```
    SchoolchildClass(2, 1),
    SchoolchildClass(3, 2),
    SchoolchildClass(4, 3),
    SchoolchildClass(5, 4),
    SchoolchildClass(6, 1),
    SchoolchildClass(6, 3)
]


def main():
    print("Рубежный контроль №1. Вариант 2 (Школьник - Класс) +
Вариант Д\n")

    print("Запрос 1 (Фамилия на 'ов' и название класса):")
    query1 = [
        (child.last_name, c.name)
        for child in schoolchildren
        for c in classes
        if child.class_id == c.id and child.last_name.endswith("ов")
    ]
    for last_name, class_name in query1:
        print(f"Школьник: {last_name} | Класс: {class_name}")
    print("-" * 50)

    print("Запрос 2 (Классы и средний балл школьников, сортировка по
возрастанию):")
    class_gpas = {
        c.name: [child.gpa for child in schoolchildren if child.class_id == c.id]
        for c in classes
    }
    query2 = [
        (c_name, sum(gpas) / len(gpas))
        for c_name, gpas in class_gpas.items()
        if len(gpas) > 0
    ]
    query2_sorted = sorted(query2, key=lambda item: item)
    for class_name, avg_gpa in query2_sorted:
        print(f"Класс: {class_name:<15} | Средний балл: {avg_gpa:.2f}")
    print("-" * 50)

    print("Запрос 3 (Классы на букву 'А' и список их школьников
[многие-ко-многим]):")
```

```
    target_classes = [c for c in classes if c.name.startswith("A")]
    for c in target_classes:
        child_ids = [sc.child_id for sc in schoolchildren_classes if sc.class_id ==
c.id]
        child_names = [child.last_name for child in schoolchildren if child.id in
child_ids]
        print(f"Класс: {c.name}")
        print(f"  Школьники: {', '.join(child_names) if child_names else 'Нет
школьников'}")


if __name__ == "__main__":
    main()
```

Пример работы программы

```
Запрос 1 (Фамилия на 'ов' и название класса):
Школьник: Иванов | Класс: Айти-класс
Школьник: Петров | Класс: Айти-класс
Школьник: Сидоров | Класс: Био-класс
Школьник: Смирнов | Класс: Гуманитарный
Школьник: Попов | Класс: Академический
--------------------------------------------------
Запрос 2 (Классы и средний балл школьников, сортировка по возрастанию):
Класс: Гуманитарный    | Средний балл: 3.50
Класс: Био-класс       | Средний балл: 4.20
Класс: Айти-класс      | Средний балл: 4.35
Класс: Академический   | Средний балл: 4.60
--------------------------------------------------
Запрос 3 (Классы на букву 'А' и список их школьников [многие-ко-многим]):
Класс: Айти-класс
  Школьники: Иванов, Петров, Попов
Класс: Академический
  Школьники: Иванов, Кузнецова, Попов
```